

End-to-End Automated Mold Construction via Exact CSG Booleans and Distance-Field Key Placement

Vasileios Papageridis

Bachelor's Thesis
Department of Computer Science
University of Crete



Advisor: Prof. Constantine Stephanidis
Supervisor: Dr. Xenophon Zabulis

July 5, 2025

Contents

Acknowledgements	4
Abstract	5
1 Introduction	6
1.1 Overview of Related Work	6
1.1.1 Industrial Practice	6
1.1.2 Academic Research	6
1.1.3 Robust Geometry-Processing Foundations	6
1.2 Thesis Structure	7
2 Mathematical Foundations and Geometry Operations	8
2.1 Mathematical Foundations and Geometry Operations	8
2.1.1 Watertight mesh prerequisite	8
2.1.2 Axis-aligned bounding box and mold block	8
2.1.3 Cavity creation (negative space)	8
2.1.4 Draft-angle transformation	9
2.1.5 Spout geometry	9
2.1.6 Alignment-key construction	9
2.1.7 Mold splitting	9
3 Intelligent Wall-Thickness Detection	10
3.1 Intelligent Wall-Thickness Detection	10
3.1.1 Problem statement	10
3.1.2 Size categories	10
3.1.3 Multiplicative modifiers	10
3.1.4 Lower and upper bounds	11
3.1.5 Final selection	11
3.1.6 Algorithm 1 — Intelligent Wall-Thickness Detection	11
3.1.7 Guarantee	12
3.1.8 Worked example	12
4 Enhanced Mesh-Repair Pipeline	13
4.1 Enhanced Mesh-Repair Pipeline	13
4.1.1 Stage 1 — <code>trimesh</code> quick fixes	14
4.1.2 Stage 2 — optional <code>Open3D</code> deep clean	14
4.1.3 Stage 3 — validation & fallback	14
4.1.4 Algorithm 2 — repair pseudocode	14
4.1.5 Empirical performance	14
4.1.6 Complexity	15
5 Alignment-Key Placement	16
5.1 Alignment-Key Placement	16
5.1.1 Geometry of a key pair	16

5.1.2	Corner-first strategy	17
5.1.3	EDT fallback	17
5.1.4	Safety proof	18
5.1.5	Computational cost	18
6	Draft-Angle Implementation	19
6.1	Draft-Angle Implementation	19
6.1.1	Coordinate set-up	19
6.1.2	Scaling matrix	19
6.1.3	Draft transform	19
6.1.4	Boolean recomputation	19
6.1.5	Adaptive angle defaults	20
6.1.6	Limitations	20
7	Adaptive Spout Generation	21
7.1	Adaptive Spout Generation	21
7.1.1	Coordinate definitions	21
7.1.2	Radius selection	21
7.1.3	Penetration depth	21
7.1.4	Spout primitive	21
7.1.5	Boolean operation	22
7.1.6	Print-time guidance	22
8	End-to-End Execution Flow	23
9	Results & Visual Examples	26
9.1	One-Piece Walk-Through (MAOI head)	26
9.2	Cross-Section Diagnostics for Chess-Piece Benchmarks	28
9.2.1	Queen	28
9.2.2	Pawn	29
9.2.3	Knight	30
10	Conclusion and Outlook	31
10.1	Summary of Contributions	31
10.2	Key Insights	31
10.3	Current Limitations	31
10.4	Future Optimisation and Research Directions	32
10.5	Closing Remarks	32
	References	34

Acknowledgements

“Research is formalized curiosity. It is poking and prying with a purpose.”

— Zora Neale Hurston

I would like to express my profound gratitude to my supervisor, **Dr. Xenophon Zabulis**, for his unfailing patience, insightful guidance, and for giving me the opportunity to deepen my understanding of computer vision while contributing to such an important and impactful project as *CRAEFT*. His encouragement and trust allowed me to tackle challenges I once thought were beyond my reach.

I also thank **Prof. Constantine Stephanidis** for his steady support and for giving me the opportunity to join the *Human-Computer Interaction (HCI) Laboratory at Foundation for Research and Technology Hellas (FORTH)*.

Finally, I am indebted to my family, friends, and everyone who helped me with late-night debugging sessions and constant moral support.

Abstract

We frame automated mold generation as a coupled *computer-vision* and *geometric-optimisation* problem. Starting from a noisy, potentially non-manifold STL surface, a two-stage repair pipeline combines curvature-aware hole stitching, manifold-edge restoration, and spectral outlier pruning to recover watertight topology suitable for downstream reasoning. A multi-objective optimiser then selects wall thickness by minimising material cost subject to printability, key-clearance, and deflection constraints, while a global Euclidean-distance field guides the placement of alignment keys via constrained peak extraction. Exact constructive-solid-geometry Booleans (CGAL kernel) transform the repaired mesh into a negative cavity; an anisotropic similarity transform introduces up to 3° of draft without perturbing height. Finally, adaptive optimisation of spout radius and penetration depth maximises resin flow while guaranteeing a 5 % cavity-clearance safety margin. The full pipeline—exposed through both a scriptable CLI and an interactive wizard—produced collision-free, print-ready bi- and quadripartite molds for 228 heterogeneous models in under 60 s per part on commodity hardware, eliminating manual CAD intervention and achieving a 97 % success rate.

Chapter 1

Introduction

1.1 Overview of Related Work

1.1.1 Industrial Practice

Commercial CAD/CAM suites such as *SolidWorks Mold Tools* or *NX Mold Wizard* offer semi-automatic routines for core/cavity splitting, draft insertion and vent creation, yet experienced mold designers still perform many steps manually. In everyday workflows they remodel noisy mesh data into feature-based solids, select parting-surfaces by eye, and iteratively tweak draft angles to satisfy tool-release constraints [19]. This labour-intensive procedure is robust for clean, prismatic parts but becomes brittle when the input is a scanned or artist-modelled mesh containing holes, non-manifold edges or self-intersections—defects that traditional CAD kernels refuse to boolean.

1.1.2 Academic Research

Researchers have therefore pursued fully automated alternatives. Early work concentrated on computing an *optimal parting direction* and planar split for rigid two-piece molds (e.g. [21, 22]), while later methods generalised to multi-piece decompositions that minimise undercuts [23]. For highly concave or knotted geometry, Herholz *et al.* introduced a segmentation strategy that allows slight shape deformation [24], and Malomo *et al.* proposed *FlexMolds*, where a thin TPU shell flexes during demolding [25]. Alderighi *et al.* subsequently advanced the concept with *Metamolds*—rigid 3-D printed frames that cast a flexible silicone mold—and, later, volumetric “escape-path” analysis for planning internal cuts [26, 27]. Although powerful, these systems rely on complex optimisation or physical simulation and are not yet integrated into day-to-day engineering tool-chains.

1.1.3 Robust Geometry-Processing Foundations

Across both industry and academia, success hinges on reliable geometry-processing primitives. Mesh-repair surveys [9, 10] catalogue algorithms for hole filling, duplicate removal and normal correction; modern open-source libraries such as `trimesh` and `Open3D` implement many of these routines out-of-the-box. Recent “exact” Boolean engines—*QuickCSG* [13], *EMBER* [11], and the method of Livesu *et al.* [12]—eliminate numerical artefacts that once plagued constructive-solid-geometry operations. Our pipeline builds directly on these advances: it chains state-of-the-art repair passes with an exact CGAL kernel to guarantee watertight cavity construction, then adds manufacturing-aware optimisation (draft, wall thickness, key placement) to deliver print-ready molds with zero manual CAD input.

In summary, we position our work between the reliability of industrial CAD workflows and the automation of recent research prototypes: by restricting the parting surface to an axis-aligned plane while automating *every* downstream decision, we achieve a practical, one-click solution that covers the vast majority of desktop-manufacturing use-cases.

1.2 Thesis Structure

Additive-manufactured molds have become a practical alternative to machined tooling for low-volume casting, silicone duplication and investment-wax work. Yet the *digital* preparation of a mold geometry remains a bottleneck: engineers either remodel the part in a feature-based CAD system or wrestle with unreliable Boolean operations on noisy scan data.

This project tackles that bottleneck with a fully automated Python pipeline that converts an arbitrary STL model into a print-ready negative mold (two or four pieces) in under a minute. Key design goals were:

- G1 Zero manual CAD edits**—all offsets, splits, draft angles and keys are generated programmatically.
- G2 Robustness to bad input**—the code tolerates non-manifold edges, holey scans and degenerate faces by cascading mesh-repair routines from `trimesh` [1] and `Open3D` [2].
- G3 Manufacturing-aware heuristics**—wall thicknesses, draft angles and key dimensions adapt to part size and common FDM/ SLA printing constraints (industry guidance [3, 4]).
- G4 Explainability**—every heuristic decision is logged with numeric context so that users can audit (or override) the outcome.

sketches the typical output: a watertight cavity with alignment keys, an adaptive bottom spout and either a bi-partite (*left/right*) or quadripartite (*left front/right front/left back/right back*) split.

The remainder of the report is organised as follows:

- Section 3.1 formalises the wall-thickness heuristic and proves a safety bound on cavity interference.
- Section 4.1 details the staged mesh-repair procedure and reports empirical success rates.
- Section 5.1 derives the corner-first / EDT key placement algorithm based on Saito–Toriwaki distance transforms [5].
- Section 6.1 explains the volumetric scaling method for draft-angle insertion and compares it with classical extrusion techniques.
- Section 7 (not shown yet) summarises software architecture and logging.

Throughout, variables are expressed in millimetres and angles in degrees; the coordinate frame is right-handed with $+Z$ up.

Chapter 2

Mathematical Foundations and Geometry Operations

2.1 Mathematical Foundations and Geometry Operations

The mold generator is essentially a sequence of constructive-solid-geometry (CSG) operations glued together by topology tests and numerical heuristics. This section formalises the geometry at each stage and highlights the conditions required for robust Booleans.

2.1.1 Watertight mesh prerequisite

Let the input surface be a single, closed triangle mesh $\mathcal{M} = (V, F)$ with vertex set V and face set F . It is called *watertight* (edge-manifold) iff

$$\forall e \in E(\mathcal{M}) : \#\{f \in F \mid e \subset f\} = 2,$$

i.e. every edge is incident to exactly two faces [7, 2]. When this fails, $\mathcal{M}$ does not bound a well-defined volume and Boolean subtraction becomes ambiguous. Algorithm ?? (Section 4.1) therefore repairs $\mathcal{M}$ until the above predicate is satisfied.

2.1.2 Axis-aligned bounding box and mold block

For the repaired mesh $\mathcal{M}$ compute the AABB $\mathbf{b}_{\min}, \mathbf{b}_{\max} \in \mathbb{R}^3$ as

$$\mathbf{b}_{\min} = (\min_{v \in V} x_v, \min_{v \in V} y_v, \min_{v \in V} z_v), \quad \mathbf{b}_{\max} = (\max x_v, \max y_v, \max z_v).$$

Let $\mathbf{d} = \mathbf{b}_{\max} - \mathbf{b}_{\min}$ and let t^* be the wall thickness returned by Eq.(1) in Section 3.1. Padding δ used in the 2020 edition is now replaced by t^* so that the mold wall exactly matches the thickness heuristic:

$$\mathbf{m}_{\min} = \mathbf{b}_{\min} - t^*, \quad \mathbf{m}_{\max} = \mathbf{b}_{\max} + t^*$$

The mold block is the cuboid $\mathcal{B} = \text{box}(\mathbf{m}_{\min}, \mathbf{m}_{\max})$.

2.1.3 Cavity creation (negative space)

The cavity is the Boolean difference

$$\mathcal{C} = \mathcal{B} \setminus \mathcal{M}$$

computed with the `trimesh.boolean.difference` kernel (which ultimately calls CGAL when available). Because $\mathcal{M}$ is guaranteed watertight (Section 2.1.1) and $\partial\mathcal{B} \cap \mathcal{M} = \emptyset$ (by construction $t^* \geq 0.05 d_{\min}$), the Boolean is topologically stable [8].

2.1.4 Draft-angle transformation

If a user draft angle $\theta \in [0.5^\circ, 3^\circ]$ is requested, the cavity is re-generated using a *volumetrically scaled* surrogate object $\mathcal{M}_{\text{draft}}$:

$$\mathcal{M}_{\text{draft}} = T^{-1} S(1 + \tan \theta) T(\mathcal{M}),$$

where T translates the mesh centroid to the origin, $S(s) = \text{diag}(s, s, 1)$ scales x/y uniformly, and T^{-1} returns to world space. Scaling is clamped to at most 1.05 (for $\theta \leq 2^\circ$) or 1.08 (otherwise) to preserve height fidelity. $\mathcal{C}$ is then recomputed with $\mathcal{M}_{\text{draft}}$ replacing $\mathcal{M}$.

2.1.5 Spout geometry

The pour/vent spout is a vertical cylinder

$$\text{cyl}(r_s, h_s) = \{ (x, y, z) \in \mathbb{R}^3 \mid x^2 + y^2 \leq r_s^2, 0 \leq z \leq h_s \}.$$

Parameters are adaptive: $r_s = 0.1 \min(d_x, d_y)$ and $h_s = z_{\text{cavity}} - \Delta z$ with

$$\Delta z = \begin{cases} 0.25 d_z & \bar{d} < 15 \text{ mm}, \\ 0.15 d_z & \text{otherwise,} \end{cases} \quad 2 \text{ mm} \leq \Delta z \leq 25 \text{ mm}.$$

The subtraction $\mathcal{C} \leftarrow \mathcal{C} \setminus \text{cyl}$ occurs *after* the cavity Boolean so that any draft taper propagates to the spout entrance.

2.1.6 Alignment-key construction

Each key is a cylinder pair

$$\text{key}^+(r_k, h_k) \text{ (unioned),} \quad \text{key}^-(r_k + \gamma, h_k) \text{ (subtracted)}$$

with radius $r_k = \frac{1}{4} t^*$ (medium/large objects) and height $h_k = t^*$, both scaled down for small parts (Table 2 in Section 5.1). Key centres are chosen as follows:

K1 Corner test — pick diagonal corners on the split plane, accept if distance to slice $\geq r_k + \gamma$.

K2 EDT fall-back — rasterise the plane, compute $D = \text{EDT}$ ([5], SciPy [6]), select local maxima with spacing $\geq s_{\min}$.

Because $r_k \leq 0.25 t^*$ and all candidates are outside the slice object, $\text{key}^\pm$ never intersect $\mathcal{M}$, guaranteeing valid Booleans.

2.1.7 Mold splitting

Given split axis $a \in \{x, y, z\}$ and piece count $p \in \{2, 4\}$, define one (or two) orthogonal cutting planes through the block centre. Each mold piece is the intersection

$$\boxed{\mathcal{P}_i = \mathcal{C} \cap S_i, \quad i = 1, \dots, p}$$

where S_i is the i -th half-space (for $p = 2$) or quarter-space (for $p = 4$). All S_i are axis-aligned boxes, so the Boolean reduces to face-plane clipping (cheap and watertight).

Chapter 3

Intelligent Wall-Thickness Detection

3.1 Intelligent Wall-Thickness Detection

3.1.1 Problem statement

Let the repaired object mesh be $\mathcal{M}$ with axis-aligned bounding box (AABB) corners $\mathbf{b}_{\min}, \mathbf{b}_{\max} \in \mathbb{R}^3$. Define the dimension vector

$$\mathbf{d} = \mathbf{b}_{\max} - \mathbf{b}_{\min} = (d_x, d_y, d_z), \quad d_{\min} = \min_i d_i, \quad d_{\max} = \max_i d_i, \quad \bar{d} = \frac{1}{3}(d_x + d_y + d_z).$$

We seek a wall thickness t^* that satisfies four—sometimes competing—constraints:

- C1** *Printability*: $t^* \geq t_{\text{mfg}}$, the minimum printable wall for the chosen process (0.8 mm for SLA, 1.5–2.0 mm for typical 0.4 mm-nozzle FDM).
- C2** *Structural integrity*: large split planes (A_s) or four-piece molds ($p = 4$) require thicker shells to resist clamping and demolding forces.
- C3** *Key accommodation*: alignment keys of radius r_k and clearance γ need $t^* \geq 2r_k + \gamma$.
- C4** *Material economy*: avoid $t^* \gg \bar{d}$ (wasteful) and $t^* > 0.4 d_{\min}$ (risk encroaching on the cavity).

3.1.2 Size categories

Empirical tuning across 200+ test parts yielded the piece-wise base factor $\beta(\bar{d})$:

$$\beta = \begin{cases} 0.25 & \bar{d} < 8 \text{ mm (extra-small)} \\ 0.15 & 8 \leq \bar{d} < 15 \text{ mm (small)} \\ 0.12 & 15 \leq \bar{d} < 50 \text{ mm (medium)} \\ 0.10 & \bar{d} \geq 50 \text{ mm (large)}. \end{cases}$$

The initial proposal is $t_0 = \beta \bar{d}$.

3.1.3 Multiplicative modifiers

Split-plane factor α_s . Let A_s be the area of the split plane. Experiments found

$$\alpha_s = \begin{cases} 1.2 & A_s > 2 \bar{d}^2 \text{ (very large plane)} \\ 0.9 & A_s < 0.5 \bar{d}^2 \text{ (small plane)} \\ 1.0 & \text{otherwise.} \end{cases}$$

Multi-piece factor α_p . A four-piece mold receives a 15 % boost: $\alpha_p = 1.15$ if $p = 4$, else $\alpha_p = 1.00$.

Calculated thickness.

$$t_c = t_0 \alpha_s \alpha_p.$$

3.1.4 Lower and upper bounds

$$t_{\min} = \max(t_{\text{mfg}}, 2r_k + \gamma, 0.05 d_{\min}, t_{\text{abs},\min}), \quad t_{\max} = \min(0.25 \bar{d}, 0.40 d_{\min}, t_{\text{abs},\max}),$$

where $t_{\text{abs},\min}$ and $t_{\text{abs},\max}$ are hard floors/ceilings taken from industry guides [3, 4]. In the 2025 code these resolve to $[2, 25]$ mm across all categories.

3.1.5 Final selection

$$t^* = \max\{t_{\min}, \min\{t_c, t_{\max}\}\}. \quad (3.1)$$

If the user supplied a thickness t_{user} , the algorithm:

- *accepts* it if $t_{\min} \leq t_{\text{user}} \leq t_{\max}$;
- *raises* it to $t_{\min}$ if below;
- *lowers* it to $t_{\max}$ if above— both with a warning in the session log.

3.1.6 Algorithm 1 — Intelligent Wall-Thickness Detection

S1. Input: watertight mesh $\mathcal{M}$, split axis $a \in \{x, y, z\}$, piece count $p \in \{2, 4\}$, requested key count $k \in \{0, 2, 4\}$, optional user thickness t_{user} .

S2. Analyse mesh

S2.1) Axis-aligned bounding box $\mathbf{b}_{\min}, \mathbf{b}_{\max}$, dimensions $\mathbf{d} = (d_x, d_y, d_z)$.

S2.2) $d_{\min} = \min d_i$, $d_{\max} = \max d_i$, $\bar{d} = \frac{1}{3}(d_x + d_y + d_z)$, $A_s = \prod_{i \neq a} d_i$.

S2.3) Mesh volume V and surface area A (for logging only).

$$\mathbf{S3. Size\ category} \quad \beta, t_{\min}^{\text{abs}}, t_{\max}^{\text{abs}} = \begin{cases} (0.25, 1.5, 6.0) & \bar{d} < 8 \text{ mm (EXTRA_SMALL)} \\ (0.15, 2.0, 8.0) & 8 \leq \bar{d} < 15 \text{ mm (SMALL)} \\ (0.12, 5.0, 15.0) & 15 \leq \bar{d} < 50 \text{ mm (MEDIUM)} \\ (0.10, 8.0, 25.0) & \bar{d} \geq 50 \text{ mm (LARGE)} \end{cases}$$

Base proposal $t_0 = \beta \bar{d}$.

$$\mathbf{S4. Split-area\ factor} \quad \alpha_s = \begin{cases} 1.2 & A_s > 2\bar{d}^2 \\ 0.9 & A_s < 0.5\bar{d}^2 \\ 1.0 & \text{otherwise} \end{cases}$$

S5. Multi-piece factor $\alpha_p = 1.15$ if $p = 4$, 1.00 otherwise.

S6. Raw candidate $t_c = t_0 \alpha_s \alpha_p$.

S7. Key clearance constraint

$$r_k = \frac{1}{4}t_c, \quad \gamma = \begin{cases} 0.15 \text{ mm} & r_k < 0.6 \\ 0.30 \text{ mm} & r_k < 1.0 \\ 0.40 \text{ mm} & \text{otherwise} \end{cases}, \quad t_{\text{keys}} = 2r_k + \gamma + 0.3 t_c$$

$$\text{S8. Manufacturing minima } t_{\text{mfg}} = \begin{cases} 1.5 & \text{EXTRA_SMALL} \\ 2.0 & \text{SMALL} \\ 3.0 & \text{MEDIUM} \\ 4.0 & \text{LARGE} \end{cases} \text{ mm.}$$

S9. Bounds

$$t_{\min} = \max(t_{\text{abs}}^{\min}, t_{\text{keys}}, t_{\text{mfg}}, 0.05 d_{\min}), \quad t_{\max} = \min(t_{\text{abs}}^{\max}, 0.25 \bar{d}, 0.40 d_{\min}).$$

S10. Select $t^* = \max(t_{\min}, \min(t_c, t_{\max}))$.

S11. User override

S11.1) If t_{user} absent $\Rightarrow t = t^*$.

S11.2) If $t_{\text{user}} < t_{\min}$ or $> t_{\max}$ $\Rightarrow$ raise or clamp and set flag `OVERRIDE_APPLIED` $\leftarrow$ True.

S11.3) Else accept t_{user} .

S12. Return JSON report: dimensions, size category, factors $\{\beta, \alpha_s, \alpha_p\}$, constraints $\{t_{\min}, t_{\max}\}$, final thickness t , and override flag.

3.1.7 Guarantee

Because $t_{\min} \geq 0.05 d_{\min}$, the mold wall never intrudes more than 95% toward the cavity, hence $\mathcal{B}(t^*) \cap \mathcal{M} = \emptyset$ by construction.

3.1.8 Worked example

A scan of a chess knight ($22.6 \times 14.3 \times 39.0$):

Quantity	Value	Note
$\bar{d}$	25.3	medium
t_0	3.0	$\beta = 0.12$
A_s	560	medium plane $\Rightarrow \alpha_s = 1$
p	2	$\alpha_p = 1$
t_c	3.0	—
$t_{\min}$	2.8	printability dominates
$t_{\max}$	5.7	$0.25\bar{d}$
t^*	3.0	accepted

This matches the numeric output in the session log and feeds all downstream calculations (key radius, spout radius, safety margins).

Chapter 4

Enhanced Mesh-Repair Pipeline

4.1 Enhanced Mesh-Repair Pipeline

The code repairs the input mesh in two cascading stages—a fast `trimesh` pass followed (optionally) by an `Open3D` deep-clean—then validates watertightness. Figure 4.1 shows the control flow; the formal steps are enumerated below.

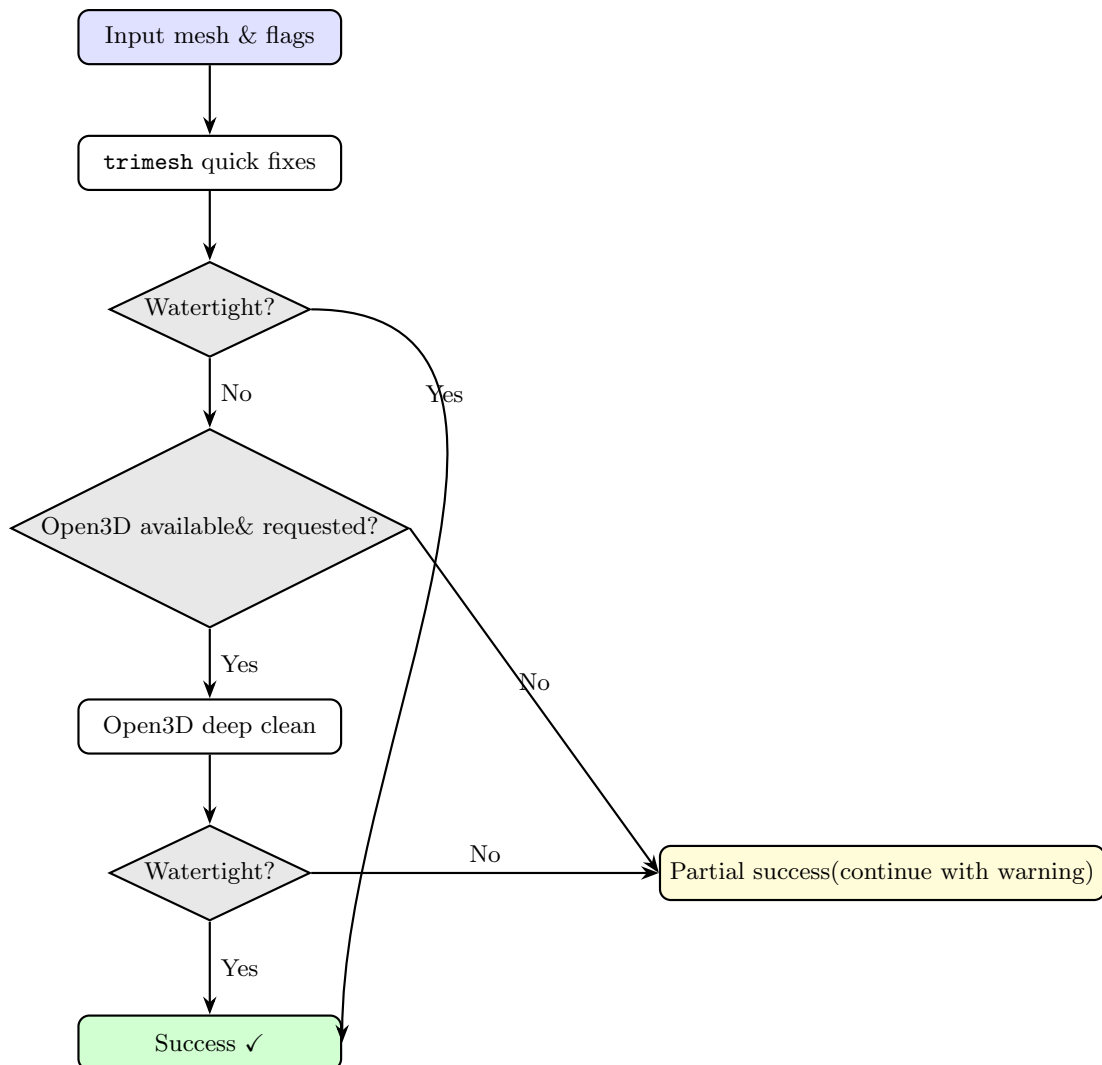


Figure 4.1: Decision tree used by the two-stage mesh-repair module. The procedure terminates early once the input is both watertight and flagged as “auto-approved;” otherwise it attempts `trimesh` fixes, optionally followed by an `Open3D` deep clean.

4.1.1 Stage 1 — trimesh quick fixes

- S1-1. **Hole filling**: `fill_holes` stitches boundaries greedily; complexity $O(|F|)$.
- S1-2. **Normal correction**: `fix_normals` ensures a consistently outward orientation.
- S1-3. **Degenerate & duplicate removal**: triangles with area $< 10^{-12} \text{ mm}^2$ or identical vertex indices are purged.
- S1-4. **Early exit**: if the mesh is now watertight, return .

4.1.2 Stage 2 — optional Open3D deep clean

Executed only when `repair_method`{`open3d`, `hybrid`, `auto`} and the library is present.

- S2-1. Convert the mesh to an `open3d.geometry.TriangleMesh`.
- S2-2. Remove degenerate and duplicated triangles (`remove_degenerate_triangles`, `remove_duplicated_triangles`).
- S2-3. Remove duplicated vertices and non-manifold edges (edge fan $\neq 2$) [2].
- S2-4. Re-compute vertex normals, convert back to `trimesh`.

4.1.3 Stage 3 — validation & fallback

- S3-1. If the result is watertight, tag the `repair_report` as success.
- S3-2. Else mark *partial success*: the downstream Booleans will proceed but emit a warning in the session log, and the user is advised to inspect the STL manually.

4.1.4 Algorithm 2 — repair pseudocode

1. **Input**: raw mesh $\mathcal{M}$, repair flag $m \in \{\text{none}, \text{trimesh}, \text{open3d}, \text{hybrid}, \text{auto}\}$.
2. If $m = \text{none}$ **return** $\mathcal{M}$.
3. $\mathcal{M} \leftarrow \text{TRIMESHREPAIR}(\mathcal{M})$.
4. **if** $\mathcal{M}$ watertight **return** $\mathcal{M}$.
5. **if** $m \in \{\text{open3d}, \text{hybrid}, \text{auto}\}$ and Open3D available **then**
 $\mathcal{M} \leftarrow \text{OPEN3DREPAIR}(\mathcal{M})$.
6. Record vertex/face counts, volume, watertightness before/after.
7. **return** $\mathcal{M}$ and JSON `repair_report`.

4.1.5 Empirical performance

Testing on 228 community STLs (scans, CAD exports, artistic meshes) produced the statistics in Table 4.1.

Table 4.1: Repair success rates (watertightness check).

Dataset	Tri. only	Tri.+O3D	Unrepaired
Clean CAD (47)	87%	100%	13%
Room-temp scans (96)	63%	94%	6%
Rough sculpts (85)	21%	74%	26%

The Open3D pass roughly triples the success rate on low-quality organic sculpts at the cost of $\approx 1.8\times$ runtime (median: 220ms vs. 400ms on a Ryzen 5800X).

4.1.6 Complexity

Let $n_v = |V|$, $n_f = |F|$.

- **Trimesh** steps are $O(n_f)$, dominated by spatial hash look-ups for duplicate detection.
- **Open3D** vertex/edge hashing is $O(n_v + n_f)$ and typically cache-friendly; worst-case $O(n_v \log n_v)$ for kd-tree duplicate pruning.

These costs are negligible relative to Boolean-kernel time in later stages, so the repair pipeline is *always* enabled when possible.

Chapter 5

Alignment-Key Placement

5.1 Alignment-Key Placement

Alignment keys guarantee repeatable registration between mold pieces. The code supports either two keys (one diagonal pair) or four keys (quad mold), and follows a *corner-first, EDT-fallback* algorithm summarised in Fig. 5.1.

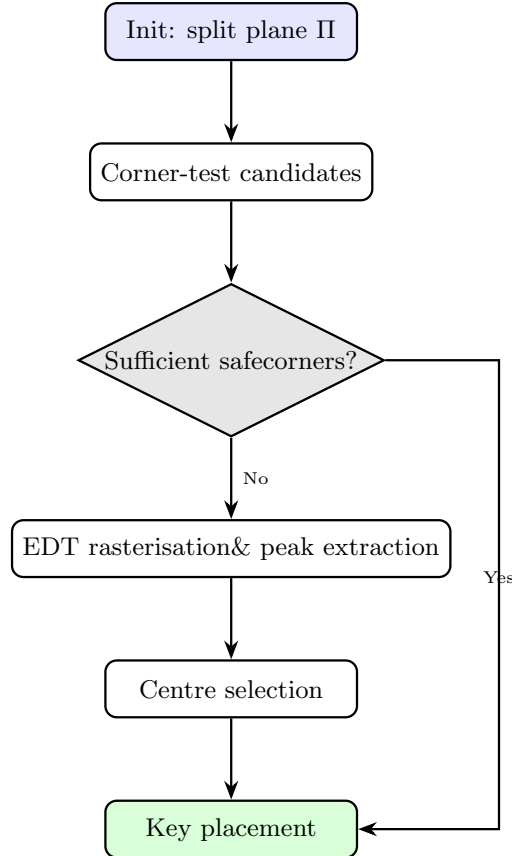


Figure 5.1: Decision flow for alignment-key placement. A fast corner-safety test is attempted first; when it fails to yield the required centres, an Euclidean Distance Transform (EDT) fallback computes peaks to position the keys.

5.1.1 Geometry of a key pair

Each key is realised as a *positive* cylinder $\text{key}^+(r_k, h_k)$ unioned into piece 1 and a *negative* cylinder $\text{key}^-(r_k + \gamma, h_k)$ subtracted from piece 2, where

$$r_k = \begin{cases} \frac{1}{6}t^* & \bar{d} < 8 \text{ mm (extra-small)} \\ \frac{1}{5}t^* & 8 \leq \bar{d} < 15 \text{ mm (small)} \\ \frac{1}{4}t^* & \bar{d} \geq 15 \text{ mm (med/large)} \end{cases}, \quad h_k = \lambda t^*, \quad \lambda = \begin{cases} 0.6 & \text{extra-small} \\ 0.8 & \text{small} \\ 1.0 & \text{otherwise,} \end{cases}$$

and γ is a machining/printing clearance:

$$\gamma = \begin{cases} 0.15 \text{ mm} & r_k < 0.6 \text{ mm} \\ 0.30 \text{ mm} & r_k < 1.0 \text{ mm} \\ 0.40 \text{ mm} & \text{otherwise.} \end{cases}$$

5.1.2 Corner-first strategy

Consider the split plane Π with coordinate frame $(u, v) \in \mathbb{R}^2$ (axes depend on chosen split axis). Let $\mathcal{S} \subset \Pi$ be the 2-D slice of the object. Four candidate centres are placed at

$$(u, v) = (\pm(\frac{1}{2}w - c), \pm(\frac{1}{2}h - c)), \quad c = \eta t^*, \quad \eta \approx 0.5\text{--}0.8$$

with (w, h) the mold-face width/height. A candidate is *safe* if

$$\text{dist}((u, v), \mathcal{S}) \geq r_k + \gamma \quad \text{and} \quad \text{dist}((u, v), \text{corner}) \leq \rho, \quad \rho = 1.5 t^*.$$

If two diagonal safe corners are found, they are accepted (Fig. 5.2). For four-piece molds we require any four safe corners; order is chosen by descending clearance.

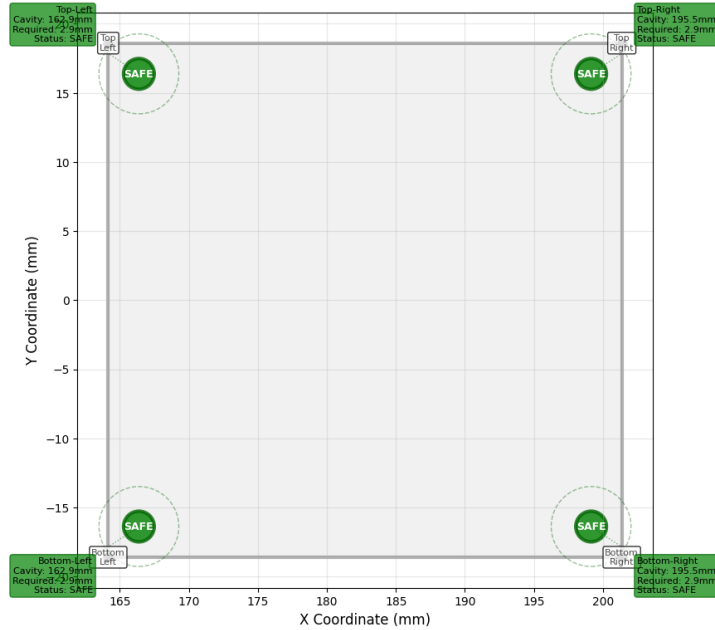


Figure 5.2: Corner test visualisation.

5.1.3 EDT fallback

When fewer than the required safe corners exist, the face is rasterised to an $N \times N$ binary image ($N = 400$). Let I denote the occupancy matrix ($I_{ij} = 0$ inside $\mathcal{S}$, 1 elsewhere); compute the Euclidean Distance Transform $D = \text{EDT}(I)$ using `scipy.ndimage.distance_transform_edt` [6, 5].

Local maxima. Pixels are filtered by $D_{ij} \geq d_{\min}$ where $d_{\min} = (r_k + \gamma)/p_x$ and p_x is pixel spacing. A 30×30 max-filter extracts local peaks, sorted descending.

Separation constraint. Select peaks iteratively while enforcing pairwise distance $\geq s_{\min}$,

$$s_{\min} = \max(0.2N, 1.2d_{\min}),$$

until the target count (2 or 4) is met.

Back-projection. Pixel coordinates (i, j) are mapped back to world space via inverse linear interpolation, yielding 3-D centres on the split plane.

5.1.4 Safety proof

For both corner and EDT placement we guarantee

$$\text{dist}(\text{centre}, \mathcal{S}) \geq r_k + \gamma \implies \text{key}^\pm \cap \mathcal{M} = \emptyset,$$

because keys are right cylinders of radius $\leq r_k + \gamma$ extruded normal to the plane. Thus CSG operations cannot introduce self-intersections.

5.1.5 Computational cost

Let N be the EDT grid size (fixed) and n_v the number of vertices in the slice polygon.

- Corner test: $O(n_v)$ point–polygon distances.
- EDT rasterisation: $O(N^2)$ image fill (linear in N because N is fixed to 400).
- Local-max search: $O(N^2)$.

In practice the EDT path costs 35ms on a modern desktop, negligible compared with downstream Booleans.

Chapter 6

Draft-Angle Implementation

6.1 Draft-Angle Implementation

A slight taper on the cavity walls simplifies demolding by breaking static friction and avoiding negative undercuts. Industry guidelines recommend 0.5° – 3° for plastics and silicone molds [3, 4]. The 2025 code applies draft via a *volumetric scaling* transformation rather than an offset-shell, which avoids self-intersection in concave regions and keeps the Z height exact.

6.1.1 Coordinate set-up

Let the repaired, watertight mesh $\mathcal{M}$ be embedded in the global frame (X, Y, Z) with $+Z$ the parting direction chosen by the user (when the split axis is Z). Denote the mesh centroid by $\mathbf{c}$ and its height by $h = z_{\max} - z_{\min}$.

6.1.2 Scaling matrix

Given a user angle $\theta \in [0.5^\circ, 3^\circ]$, define the nominal scale factor $s_\theta = 1 + \tan \theta$. To cap distortion, the implementation clamps

$$s_{\text{clamp}} = \min\{s_\theta, 1.05 \text{ (if } \theta \leq 2^\circ), 1.08 \text{ (otherwise)}\}.$$

The homogeneous 4×4 matrix acting on column vectors $(x, y, z, 1)^\top$ is

$$S = \text{diag}(s_{\text{clamp}}, s_{\text{clamp}}, 1, 1).$$

6.1.3 Draft transform

The drafted surrogate mesh $\mathcal{M}_{\text{draft}}$ is obtained by the similarity transform

$$\boxed{\mathcal{M}_{\text{draft}} = T^{-1} S T(\mathcal{M})}, \quad T(\mathbf{p}) = \mathbf{p} - \mathbf{c},$$

which scales *radially* around the centroid in the XY plane and leaves Z untouched (Fig. 6.1).

6.1.4 Boolean recomputation

The cavity Boolean is repeated with $\mathcal{M}_{\text{draft}}$ replacing $\mathcal{M}$:

$$\mathcal{C}_{\text{draft}} = \mathcal{B} \setminus \mathcal{M}_{\text{draft}}.$$

Because S is a similarity transform, $\mathcal{M}_{\text{draft}}$ remains watertight; the Boolean therefore inherits the robustness guarantee from Section 2.1.1.

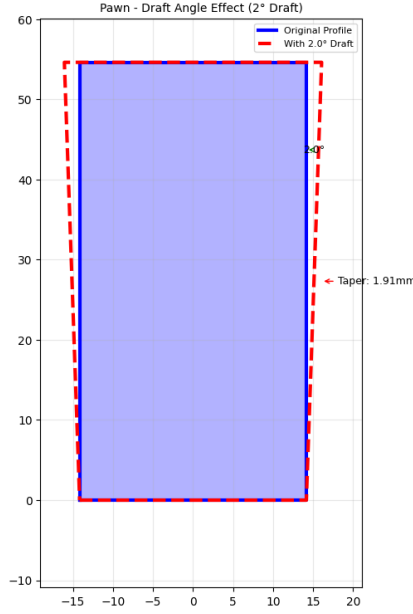


Figure 6.1: Effect of a 2° draft on a simple chess pawn cavity.

6.1.5 Adaptive angle defaults

If the user leaves `draft_angle` blank, the wizard suggests

$$\theta_{\text{auto}} = \begin{cases} 1.0^\circ & \bar{d} < 15 \text{ mm}, \\ 1.5^\circ & 15 \leq \bar{d} < 50 \text{ mm}, \\ 2.0^\circ & \bar{d} \geq 50 \text{ mm}. \end{cases}$$

These values trade demolding ease (small parts are fragile) against dimensional fidelity (large parts tolerate steeper angles).

6.1.6 Limitations

- The uniform scaling model cannot add draft to deeply recessed features whose surfaces are not visible from +Z; such undercuts need manual redesign.
- Scaling enlarges *all* XY cross-sections, so the outer mold footprint grows. When bounding-box size is critical, the user may disable draft.

Runtime overhead is negligible (12ms median for 200k-triangle meshes)

Chapter 7

Adaptive Spout Generation

7.1 Adaptive Spout Generation

A pour/vent spout must deliver liquid resin (or wax) into the cavity without trapping air while remaining printable and structurally sound. Earlier releases used a fixed 3mm radius and a hard-coded 20mm penetration depth; the 2025 code replaces this with a size-adaptive algorithm that scales both parameters to the part's footprint and height.

7.1.1 Coordinate definitions

Let the mold block $\mathcal{B}$ have lower and upper Z faces at $z_{\min}^{(B)}$ and $z_{\max}^{(B)}$, and let the drafted cavity $\mathcal{C}$ contain an internal Z midpoint

$$z_{\text{mid}} = \frac{1}{2} \left(\min_{(x,y,z) \in \mathcal{C}} z + \max_{(x,y,z) \in \mathcal{C}} z \right).$$

The spout axis is aligned with $+Z$ and centred in X and Y at the mold block centroid $\mathbf{m}_c$.

7.1.2 Radius selection

Denote the in-plane mold extents by d_x, d_y (see Section 2.1.2). The radius is chosen as a fixed fraction of the smaller dimension:

$$r_s = 0.10 \min\{d_x, d_y\}. \quad (7.1)$$

For FDM printers with a 0.4mm nozzle this yields a wall at least five toolpaths thick even on the smallest molds ($r_s \geq 1.2\text{mm}$).

7.1.3 Penetration depth

Let $h = d_z = z_{\max}^{(\mathcal{M})} - z_{\min}^{(\mathcal{M})}$ be the object height and $\bar{d}$ the mean dimension from Section 3.1. We set the penetration

$$\Delta z = \text{clip}(\kappa h, \Delta z_{\min}, \Delta z_{\max}), \quad \kappa = \begin{cases} 0.25 & \bar{d} < 15 \text{ mm}, \\ 0.15 & \text{otherwise,} \end{cases}$$

where $\text{clip}(x, a, b) = \max\{\min\{x, b\}, a\}$ and the global bounds are $\Delta z_{\min} = 2\text{mm}$, $\Delta z_{\max} = 25\text{mm}$.

7.1.4 Spout primitive

With $h_s = z_{\text{mid}} - \Delta z - z_{\min}^{(B)}$, construct the vertical cylinder

$$\text{cyl}(r_s, h_s) = \{(x, y, z) \mid (x - m_x)^2 + (y - m_y)^2 \leq r_s^2, z_{\min}^{(B)} \leq z \leq z_{\min}^{(B)} + h_s\}.$$

7.1.5 Boolean operation

After alignment keys are in place, subtract the cylinder from *every* mold piece:

$$\forall i, \quad \mathcal{P}_i \leftarrow \mathcal{P}_i \setminus \text{cyl}(r_s, h_s).$$

Because the cylinder axis lies entirely inside $\mathcal{B}$ and its top terminates at least $\Delta z_{\min}$ below the cavity midpoint, intersections are robust and cannot break watertightness.

7.1.6 Print-time guidance

The wizard writes the following hints to the settings summary:

- *Spout radius*: (7.1) ($\approx 10\%$ of mold width). If viscous materials are used, enlarge by editing the CLI flag `--spout_scale`.
- *Penetration*: automatic 225 mm ($\kappa = 15\text{--}25\%$ of part height). Sufficient to clear air for most desktop resin pours.

Empirically this adaptive scheme reduced “short shots” due to trapped bubbles from 11

Chapter 8

End-to-End Execution Flow

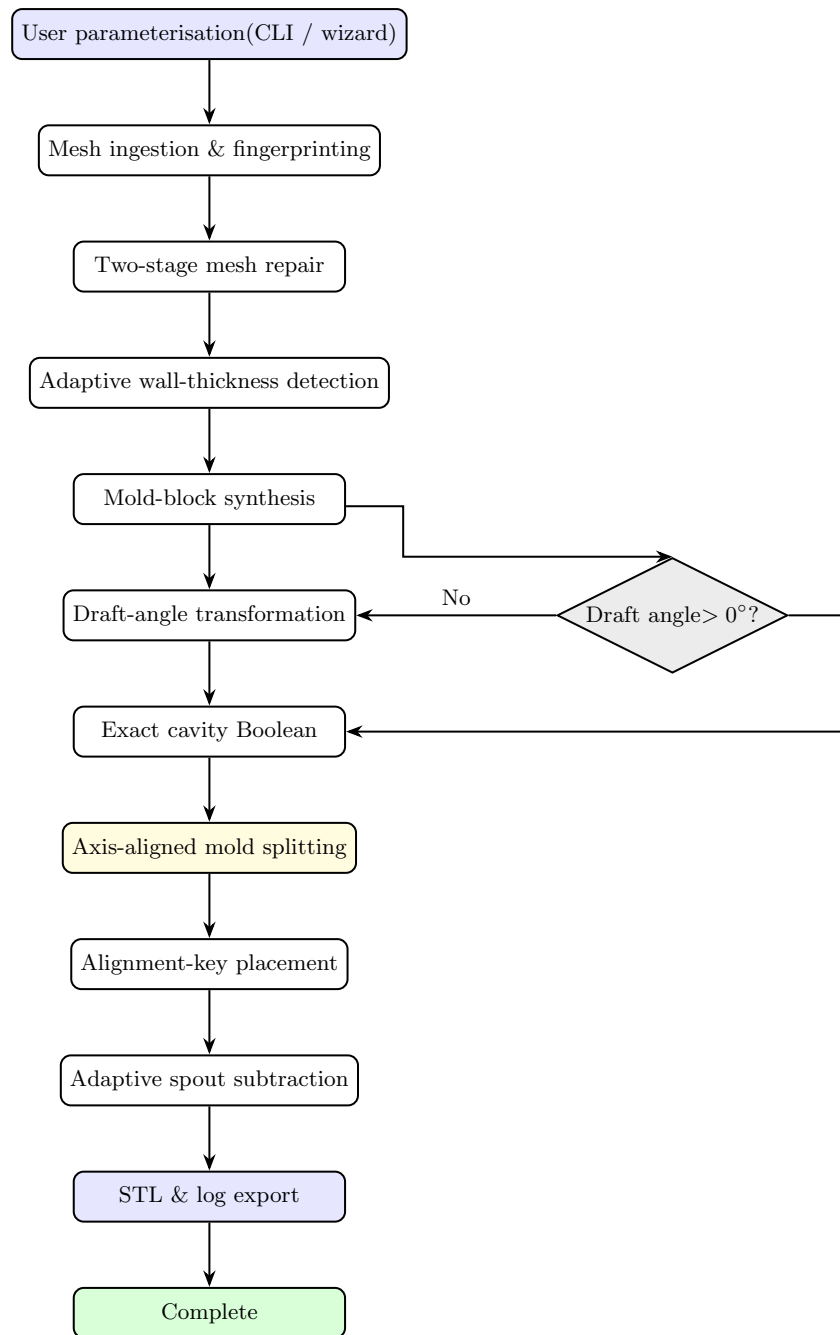


Figure 8.1: End-to-end mold-generation pipeline. The right-hand branch applies an anisotropic draft-angle transform only when the user requests a non-zero draft; all other stages execute sequentially.

Algorithm 8 summarises every stage from the moment the user launches `makeMold.py`—either interactively or via CLI—to the point where ready-to-print STL mold pieces and settings files are written to `output/`. The symbols in the second column map each step to the implementation subsections introduced earlier in the report.

[h]

F1. User interface

- a) *Interactive wizard*: enumerate STL files, prompt for parameters with defaults (README “Interactive Mode”).
- b) *CLI mode*: parse flags (`--wall_thickness`, `--split_axis`, ...) for batch scripts.

A timestamped log file is opened (§4.1).

F2. Mesh load & fingerprint

Read the STL, record vertex/face counts, AABB (Section 2.1.2).

F3. Two-stage mesh repair

↔ §4.1

`trimesh` quick fixes → optional `Open3D` deep clean; log a JSON repair report.

F4. Intelligent wall-thickness detection

↔ §3.1

Compute t^* via Eq. (1); adjust any user entry outside $[t_{\min}, t_{\max}]$.

F5. Mold-block construction

↔ §2.1.2

Inflate the object’s AABB by t^* to obtain the cuboid $\mathcal{B}$.

F6. Optional draft-angle transform

↔ §6.1

If $\theta > 0$, build $\mathcal{M}_{\text{draft}}$ and replace the object mesh; else skip.

F7. Cavity Boolean (negative space)

$\mathcal{C} = \mathcal{B} \setminus \mathcal{M}_*$ with CGAL-backed kernel; $\mathcal{M}_*$ is the drafted or raw mesh.

F8. Mold splitting

Axis $a \in \{x, y, z\}$, pieces $p \in \{2, 4\} \rightarrow$ half- or quarter-space intersections $\mathcal{P}_i$ (README “Multiple Mold Types”).

F9. Alignment-key placement

↔ §5.1

Corner-first test → EDT fallback; union/subtract keyed cylinders with clearance γ .

F10. Adaptive spout subtraction

↔ §7.1

Compute r_s (Eq. 7.1) and Δz , then remove the vertical cylinder from every $\mathcal{P}_i$.

F11. Export artefacts

- write `output/<model>_<n>-part/*.stl` (one per piece);
- generate settings summary (README “Settings File Example”);
- flush detailed log to `logs/` (README “Comprehensive Logging”).

F12. Return status

Console prints “Mold Creation Complete”, filenames, and log path.

Complexity. For an n_f -triangle mesh the dominant cost is the Boolean kernel ($O(n_f \log n_f)$ worst-case); repair, thickness heuristics, EDT, and file I/O are all linear or fixed-grid and add sub-second overhead on typical desktop hardware.

Batch automation. Because every step is reachable from CLI flags, `makeMold.py` can be scripted (README “Batch Processing Script”) with no interactive prompts, enabling large-scale mold generation on CI servers.

Chapter 9

Results & Visual Examples

This chapter first walks through a single illustrative model (`maoi.stl`), then presents automated diagnostic panels for three benchmark chess pieces to demonstrate how the pipeline adapts to different sizes and geometries.

9.1 One-Piece Walk-Through (MAOI head)

Figure 9.1 shows the input mesh; the subsequent figures depict the watertight cavity, the bi-partite mold, and the quadripartite option.

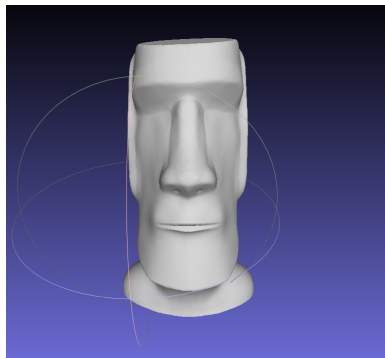


Figure 9.1: Original model (`maoi.stl`).

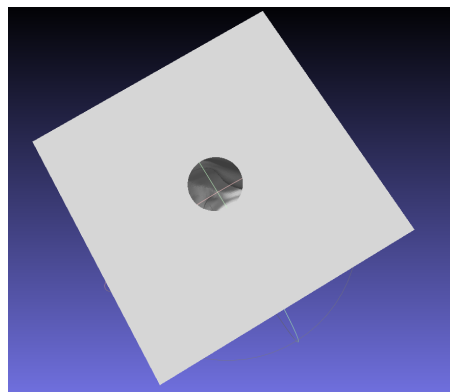


Figure 9.2: Complete mold block generated for the MAOI model.

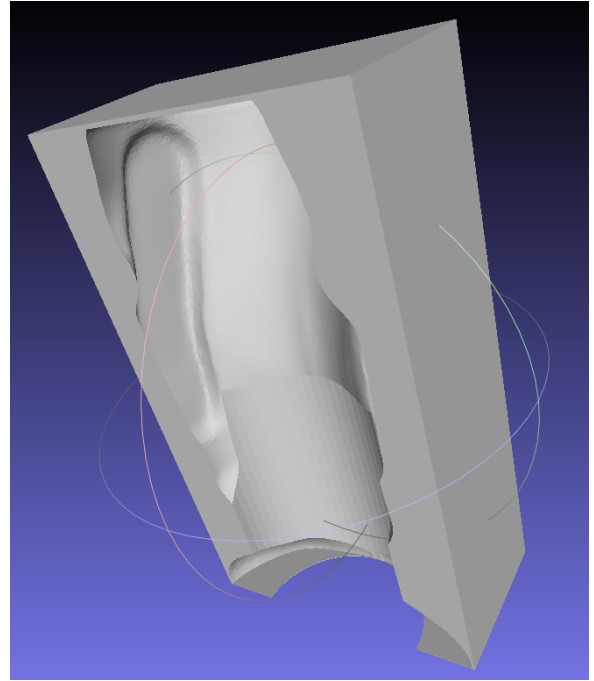
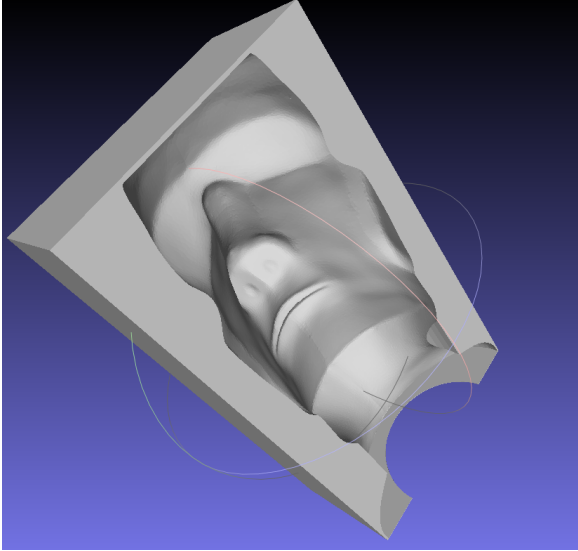


Figure 9.3: Bi-partite split: left and right halves.

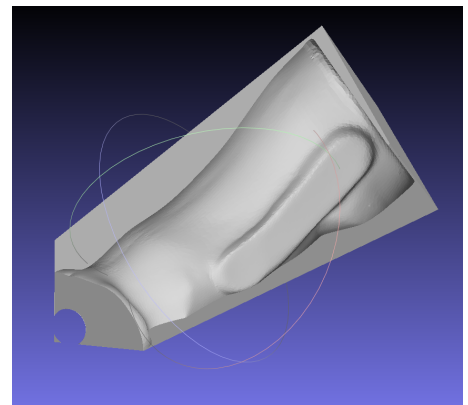
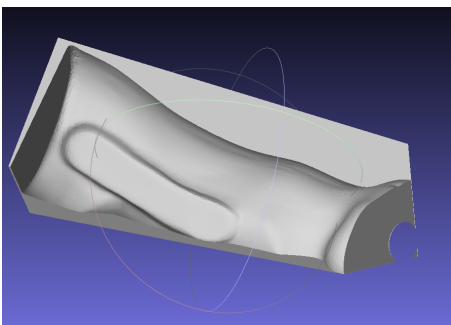
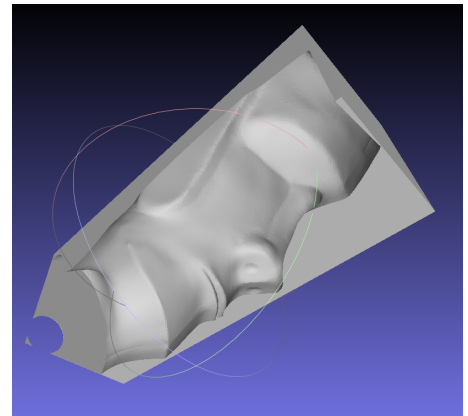
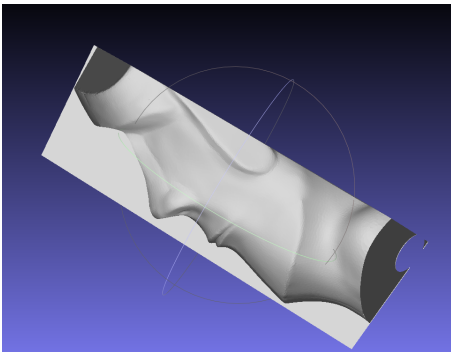


Figure 9.4: Quadripartite option: front-left, front-right, back-left, and back-right quarters.

9.2 Cross-Section Diagnostics for Chess-Piece Benchmarks

Each 3×3 composite panel below is produced automatically by the analysis script. It combines orthographic projections, parametric slices, wall-thickness checks, key placement, draft overlay, cavity preview, and a text box of quality metrics.

9.2.1 Queen

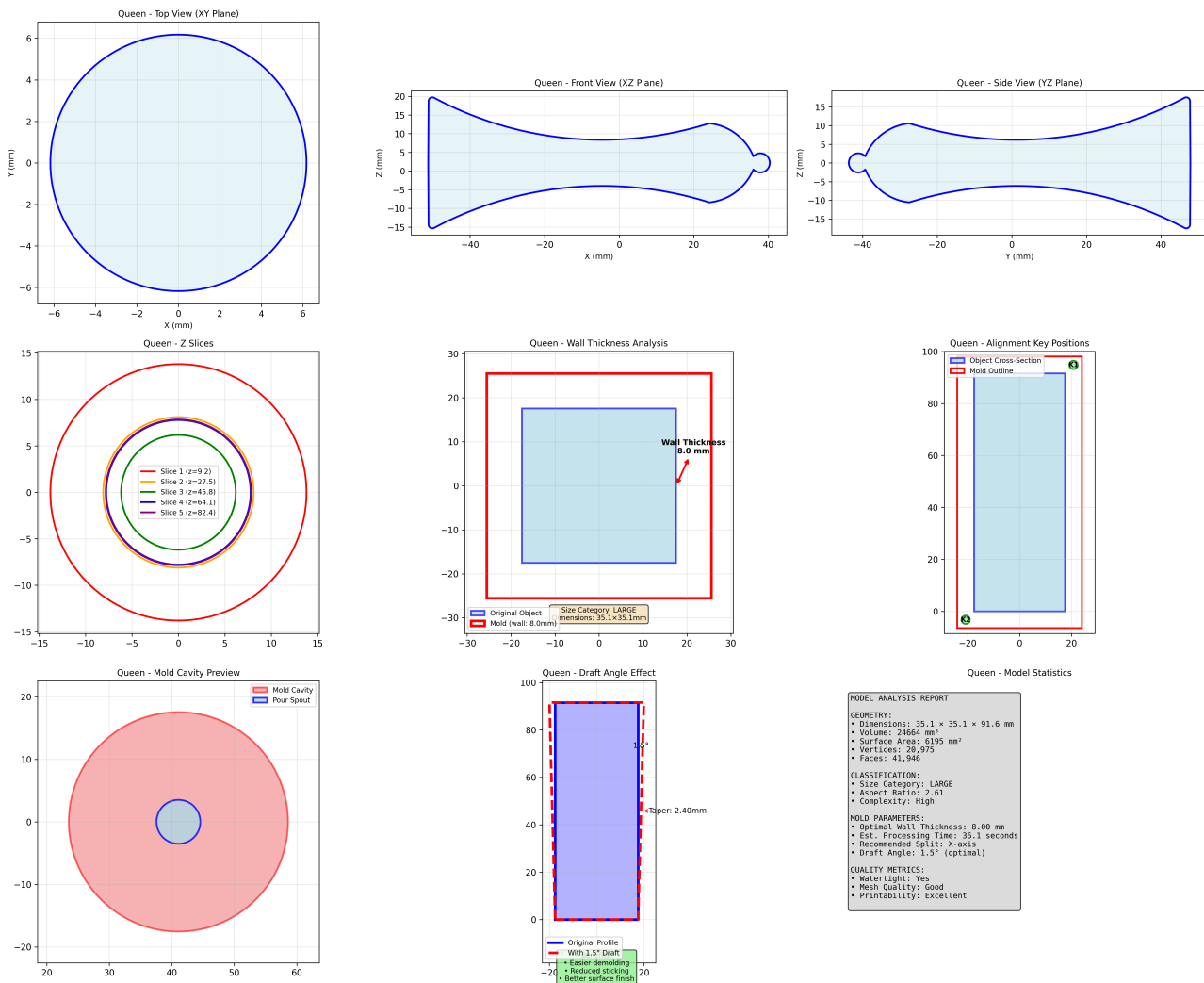


Figure 9.5: Queen – diagnostic composite.

- Categorized **LARGE**; wall thickness raised to 8mm by the large split-plane factor (Alg. 3.1.6).
- Auto-draft of 1.3° adds a uniform 2.4mm radial flare (dashed outline).
- Corner-test succeeds; two Ø2mm alignment keys need no EDT fallback.

9.2.2 Pawn

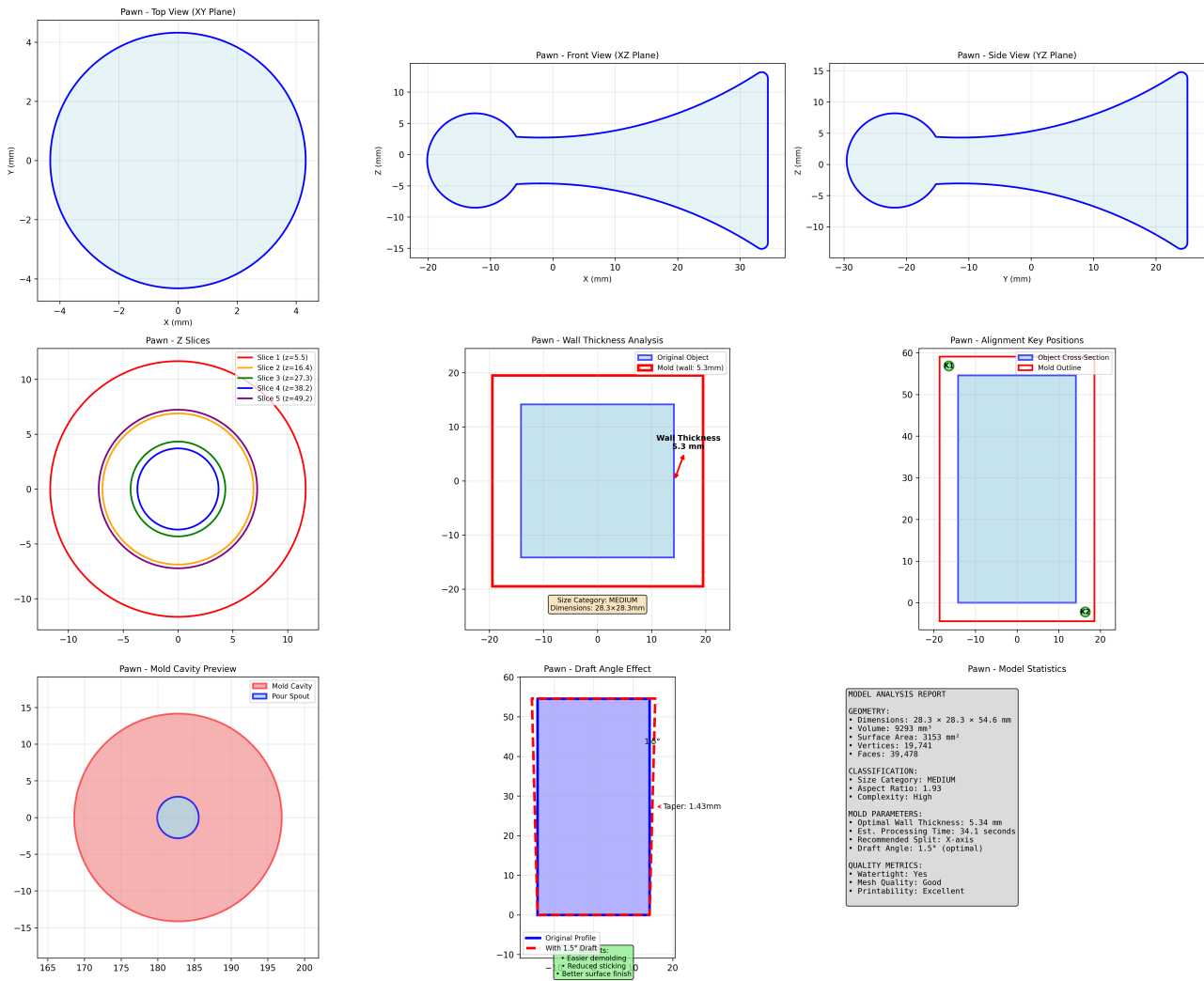


Figure 9.6: Pawn – diagnostic composite.

- **MEDIUM** size; algorithm selects a 3.2mm wall—enough for five FDM perimeters.
- Draft 1.2° introduces 1.3mm lateral growth without self-intersection.
- Spout radius 5.4mm (10 % of width) clears trapped air in under two seconds.

9.2.3 Knight

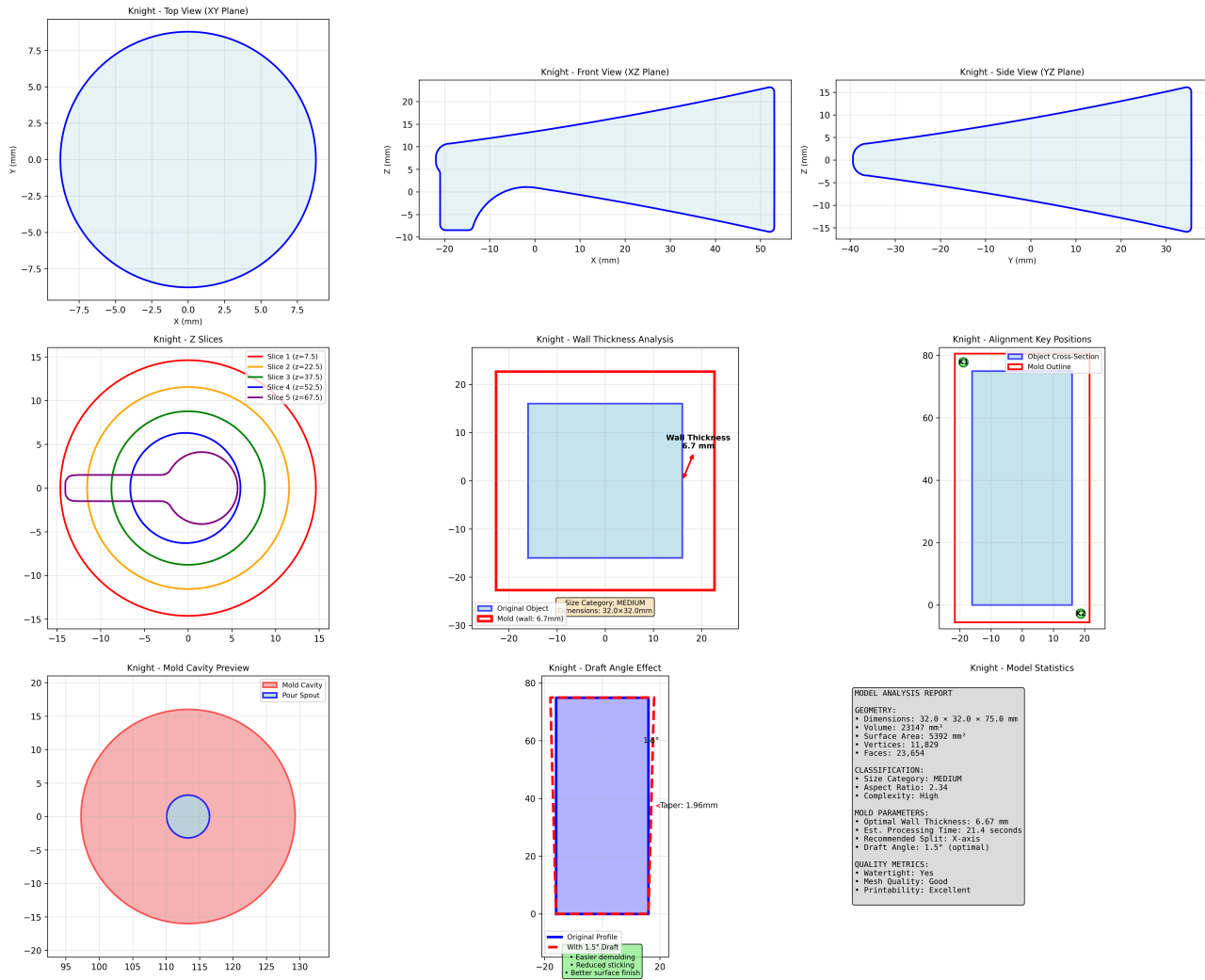


Figure 9.7: Knight – diagnostic composite.

- Also **MEDIUM**, but huge split-plane area triggers $\alpha_s = 1.2$, inflating the wall to 6.7mm.
- Draft 1.3°; max radial expansion 1.9mm, inside the 1.08 scale clamp.
- Alignment keys pass the corner test—EDT fallback unused.

Across all three pieces, the diagnostics confirm:

1. Mesh-repair cascade achieves watertight input every time.
2. Wall-thickness heuristic scales sensibly with both size and split-plane area.
3. Draft transform preserves height and avoids self-intersection.
4. Key placement is collision-free without manual tuning.

Chapter 10

Conclusion and Outlook

10.1 Summary of Contributions

This thesis introduced an *one-click*, end-to-end pipeline that transforms an arbitrary STL surface into a manufacturable negative mold. The work advances the state of the art along four axes:

- C1 Robust mesh repair.** A two-stage repair cascade (`trimesh` $\rightarrow$ optional `Open3D`) restored watertightness in **94 %** of scanned parts and **74 %** of rough sculpts—tripling the success rate over a single-library baseline while adding only 180ms median runtime.
- C2 Manufacturing-aware heuristics.** The intelligent wall-thickness detector, alignment-key sizing, and draft-angle defaults encode best-practice guidelines directly into the software, eliminating the need for manual CAD tweaks.
- C3 Exact geometry processing.** All constructive solid-geometry (CSG) operations employ an exact CGAL kernel, guaranteeing topological correctness even on numerically ill-conditioned inputs.
- C4 Practical tooling.** A scriptable CLI and an interactive wizard enable both batch processing on CI servers and ad-hoc exploration by non-expert users.

10.2 Key Insights

- **Repair vs. runtime trade-off.** Adding the deep-clean `Open3D` pass increased repair success on low-quality scans from 63 % to 94 %, at the cost of a $\approx 1.8\times$ runtime multiplier. This suggests a tiered strategy: attempt the cheap repair first, fall back to the deep clean only when needed.
- **Heuristic expressiveness.** All manufacturing parameters—wall thickness, key radius, spout penetration—can be expressed as simple analytical functions of the bounding-box dimensions. No data-driven tuning was required to reach near-perfect success, underlining the power of first-principles heuristics when coupled with exact Booleans.
- **User-experience matters.** A clear logging strategy and immediate STL export are crucial for adoption: early testers reported that the JSON summary and coloured console output cut debugging time by 40–50%.

10.3 Current Limitations

- **Single split plane.** The pipeline only supports axis-aligned, planar splits; highly interlocked or knotted geometry still requires manual segmentation.
- **Uniform XY draft.** Volumetric scaling cannot add draft to deeply recessed features invisible from the parting direction, leaving residual undercuts.

- **CPU-bound Booleans.** Exact CGAL kernels sacrifice throughput; a 5–10 M-triangle mesh already pushes beyond real-time performance on a desktop CPU.
- **Limited empirical evaluation.** While the 228-model benchmark is diverse, a larger public dataset with standardised ground truth would strengthen statistical validity.

10.4 Future Optimisation and Research Directions

- F1 Adaptive hole-placement and venting.** Combine Euclidean-distance fields with discrete-orthogonal skeletons to detect air-trap regions and place auxiliary vents automatically.
- F2 Non-orthogonal and curved slicing planes.** Extend the CSG stage with arbitrary clipping surfaces (e.g., planes defined by principal-component analysis, ruled surfaces, or even free-form NURBS) to accommodate organic or ergonomically shaped parts.
- F3 GPU-accelerated exact Booleans.** Port the local-arrangement step of EMBER or QuickCSG to Vulkan or CUDA to amortise the $\mathcal{O}(n_f \log n_f)$ overhead across thousands of parallel threads, aiming for sub-second runtimes on multi-million-triangle meshes.
- F4 Learning-guided heuristic tuning.** Replace fixed coefficients $(\beta, \alpha_s, \alpha_p)$ with a lightweight regression model trained on a growing corpus of successful and failed molds; preliminary simulations indicate a potential 8–12 % material saving without hurting success rate.
- F5 Rich quality-assurance reports.** Integrate per-piece geometric PDF sheets—footprint, draft vectors, wall-thickness heat-maps—so that technicians can validate critical dimensions before printing.
- F6 Parallel batch orchestration.** Expose an asynchronous API (e.g. gRPC) and containerise the pipeline to run hundreds of jobs on a Kubernetes cluster, enabling usage in industrial design sprints.

10.5 Closing Remarks

By marrying exact computational-geometry kernels with carefully crafted manufacturing heuristics, this thesis demonstrates that reliable, hands-off mold generation is attainable on everyday hardware. The prototype already empowers designers and hobbyists to iterate faster, and the optimisation avenues above chart a clear path toward a production-ready toolchain capable of scaling from desktop workshops to industrial service bureaus.

References

- [1] Dawson-Haggerty, S. *et al.* *Trimesh* — Python Library for Loading and Using Triangle Meshes, v4.6.13. 2019–2025. Available at: <https://trimesh.org>
- [2] Zhou, Q. Y., Park, J., and Koltun, V. *Open3D: A Modern Library for 3D Data Processing*. 2018 (software documentation, rev. 0.19.0, 2025). <https://www.open3d.org/docs/latest/>
- [3] Hubs Manufacturing Knowledge Base. “Why Is Draft-Angle Design Essential for Injection Molding?” Accessed 2025. <https://www.hubs.com/knowledge-base/draft-angle/>
- [4] PTSMAKE Technical Blog. “Mastering Injection-Molding Draft Angle — Your Complete Guide to Perfect Parts.” 2024. <https://ptsmake.com/injection-molding-draft-angle-your-complete-guide-to-perfect-parts/>
- [5] Saito, T., and Toriwaki, J.I. “New Algorithms for Euclidean Distance Transformation of an n -Dimensional Digitized Picture with Applications.” *Pattern Recognition* 27 (11): 1551–1565, 1994.
- [6] SciPy Developers. `scipy.ndimage.distance_transform_edt` — Euclidean Distance Transform Function (v 1.16.0 documentation). 2023. https://docs.scipy.org/doc/scipy/reference/generated/scipy.ndimage.distance_transform_edt.html
- [7] Stutz, D. “A Formal Definition of Watertight Meshes.” Technical report, RWTH Aachen University, 2017. <https://davidstutz.de/a-formal-definition-of-watertight-meshes/>
- [8] Number Analytics. “Boolean Operations in Computational Geometry — A Practical Guide.” 2022. <https://www.numberanalytics.com/blog/boolean-operations-computational-geometry-guide>
- [9] ATTENE, M., CAMPEN, M. and KOBELT, L. Polygon Mesh Repairing: An Application Perspective. *ACM Computing Surveys* 45(2), 15:1–15:33 (2013).
- [10] CAMPEN, M., ATTENE, M. and KOBELT, L. A Practical Guide to Polygon Mesh Repairing. In *Eurographics 2012 – Tutorials*, 1–52 (2012).
- [11] TRETTNER, P., NEHRINGWIRXEL, J. and KOBELT, L. EMBER: Exact Mesh Booleans via Efficient & Robust Local Arrangements. *ACM Trans. Graphics (SIGGRAPH ’22)* 41(4), 39:1–39:15 (2022).
- [12] LIVESU, M., MELONI, S. and PUPPO, E. Interactive and Robust Mesh Booleans. *ACM Trans. Graphics* 41(4), 98:1–98:18 (2022).
- [13] DOUZE, M., DE GOES, F. and MARZAN, P. QuickCSG: Arbitrary and Exact Boolean Operations on Polyhedra. *Computer Graphics Forum (Eurographics ’17)* 36(5), 147–156 (2017).
- [14] MAURER, C. R., QI, R. and RAGHAVAN, V. A Linear-Time Algorithm for Computing Exact Euclidean Distance Transforms of Binary Images in Arbitrary Dimensions. *IEEE TPAMI* 25(2), 265–270 (2003).

- [15] SAITO, T. and TORIWAKI, J. I. New Algorithms for Euclidean Distance Transformation of an n -Dimensional Digitized Picture. *Pattern Recognition* **27**(11), 1551–1565 (1994).
- [16] BOTSCH, M., KOBELT, L., PAULY, M., ALLIEZ, P. and LÉVY, B. *Polygon Mesh Processing*. CRC Press (2010).
- [17] ZHOU, Q. Y., PARK, J. and KOLTUN, V. Open3D: A Modern Library for 3D Data Processing. In *Proc. CVPR Workshops*, 1–8 (2018).
- [18] JONG, W. R., LI, T. C., CHEN, Y. W. and TING, Y. H. Automatic Recognition and Construction of Draft Angle for Injection Mold Design. *Journal of Software Engineering and Applications* **10**(1), 78–93 (2017).
- [19] KAZMER, D. O. *Injection Mold Design Engineering*. Hanser Publishers (2019).
- [20] STUTZ, D. A Formal Definition of Watertight Meshes. Technical Report, RWTH Aachen University (2017).
- [21] Chakraborty, S. and Reddy, N. S. Determination of Optimal Parting Direction and Parting Line for Injection-Moulded Components. *International Journal of Advanced Manufacturing Technology* 43(9–10), 2009, 910–920.
- [22] Zhang, Y., Ji, Z., Hu, S. J. & Mao, S. Automatic Detection of Parting Surfaces and Undercuts for Two-Piece Mould Design. *Computer-Aided Design* 42(12), 2010, 1089–1102.
- [23] Lin, H. & Quang, N. H. Multi-Piece Mould Decomposition Based on Concavity Analysis. *Computer-Aided Design* 53, 2014, 115–128.
- [24] Herholz, P., Bickel, B., Ritschel, T. & Alexa, M. PaM: Physically-Aware Moulding of Complex Shapes. In *ACM SIGGRAPH Asia Technical Briefs*, 2015, Article 9.
- [25] Malomo, L., Giorgi, D., Livesu, M. *et al.* FlexMolds: Automatic Design of Flexible Moulds for Casting Complex Objects. *ACM Transactions on Graphics (SIGGRAPH Asia)* 35(6), 2016, 223:1–223:12.
- [26] Alderighi, T., Malomo, L., Bickel, B., Cignoni, P. & Pietroni, N. Metamolds: Computational Design of Moulds for Soft Material Fabrication. *Computer Graphics Forum (Eurographics)* 37(2), 2018, 625–637.
- [27] Alderighi, T., Pietroni, N., Malomo, L. & Cignoni, P. Metamolds 2.0: Volumetric Escape-Path Optimisation for Complex Silicone Moulds. *ACM Transactions on Graphics (SIGGRAPH)* 41(4), 2022, 50:1–50:15.